



Data Lakehouse Architecture

AICB-P2T2 · Ngày 18 · Chương 4: Hạ Tầng

Giảng viên

VinUniversity · Phase 2 · AI Infrastructure Track · Tuần 4

HÃY SUY NGHĨ...

““Đổ tất cả vào S3” — works ở 10 GB, ác mộng ở 10 TB, production outage ở 10 PB.

Lakehouse = ACID + cheap storage + AI workloads.

Câu trả lời cho 3 era: Traditional, ML, LLM.”

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Evolution + 3 Eras (Trad / ML / LLM)
2. Delta Lake: ACID + Deletion Vectors + CDF
3. Time Travel & Data Versioning
4. Apache Iceberg: Hidden Partitioning, v3
5. Query Engines (Spark/Trino/DuckDB)
6. Storage Optimization & Anti-Patterns
7. Format Internals & Performance Tuning
8. Lakehouse cho AI/ML Workloads
9. Streaming & CDC Ingestion
10. Industrial Deep Dive (AI Thực Chiến)
11. Production Ops (Catalog, DQ, Lineage, Security, FinOps)
12. Demo + Lab repo (notebooks 01–04)

Sau buổi học này, bạn sẽ:

1. Hiểu evolution storage qua 3 era: **Traditional** → **ML** → **LLM**
2. Triển khai Delta/Iceberg: ACID, time travel, deletion vectors, CDF
3. So sánh Delta vs Iceberg vs Hudi → chọn theo workload (append / mutate / multimodal)
4. Thiết kế medallion Bronze/Silver/Gold cho LLM observability + RAG corpus
5. Áp dụng production ops: catalog, data contracts, lineage, FinOps

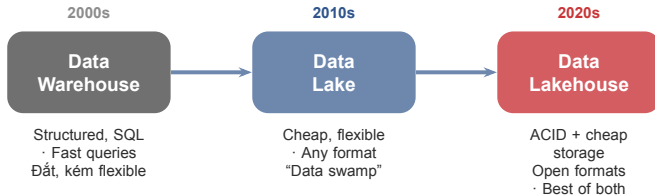
Evolution + 3 Eras → Delta + Iceberg → Storage & Performance → AI/LLM Workloads
→ Industrial Cases → Production Ops → Demo

Lakehouse Bronze/Silver/Gold + Delta ACID + time travel + benchmark — 1-to-1 với notebooks trong lab repo.

- **NB1** — Delta table + schema enforcement + `schema_mode="merge"` evolution; `_delta_log/` JSON visible
- **NB2** — OPTIMIZE + Z-ORDER: $\text{speedup} \geq 3\times$ hoặc $\text{files-pruned} \geq 10\times$ (min/max stats)
- **NB3** — MERGE 100K + RESTORE rollback; `history()` hiển thị ≥ 5 versions (kể cả RESTORE)
- **NB4** — Medallion Bronze→Silver→Gold: dedup quan sát được (Silver < Bronze) + Gold p50/p95/cost qua ≥ 7 ngày

Lab repo: github.com/VinUni-AI20k/Day18-Track2-Lakehouse-Lab — 2 paths: lightweight (delta-rs+DuckDB, ~80 MB) hoặc Spark/Docker.

Bonus (optional, không tính điểm): BONUS-CHALLENGE.md — thiết kế lakehouse cho 1 hard problem (LLM obs, Decree-13 CDC, multimodal RAG, FinOps tier, catalog migration, ...).



Key Innovation: metadata layer biến S3 object storage thành transactional store.

Enablers: Open table formats (Delta/Iceberg/Hudi) + cheap object storage (S3) + query engines (Spark/DuckDB/Trino).

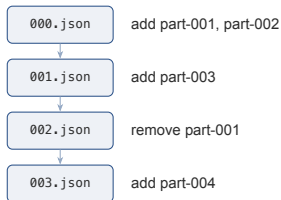
Aspect	Traditional (1990s–2010)	ML Era (~2012–2022)	LLM Era (2022+)
Workload	OLTP txn, BI reports	Feature eng, batch train/infer	Pretraining (T tokens), fine-tune, RAG, eval
Data shape	Tabular, 3NF normalized	Tabular + semi-structured (JSON)	Text + multimodal + embeddings
Volume	GB–TB	TB–PB	PB+ raw, 10^{12} + tokens, billions vectors
Latency	ms (txn), hours (BI)	Min–hours (batch ML)	Hours (train) + sub-100 ms (RAG)
Schema	Schema-on-write (rigid)	Schema-on-read or hybrid	Hybrid + lineage + contracts + provenance
Compliance	SOX, PCI, GDPR	+ bias audit, fairness	+ training data provenance, copyright, hallucination
Failure mode	Schema drift, deadlock	Data drift, train/serve skew	Dedup leak, license violation, hallucination

Pattern: mỗi era thêm *một class data mới* mà era trước không có (semi-structured → multimodal+embeddings).
Storage stack **mở rộng**, không thay thế.

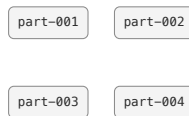
Tech	Traditional	ML Era	LLM Era
OLTP DB (Postgres)	Primary system	App backend; CDC source for ML	App backend; CDC; user feedback
Warehouse (Snowflake)	BI reporting	Feature aggregation + serving	BI dashboards on LLM telemetry
Data Lake (S3 + Hive)	—	Raw events, training datasets	Raw crawls, multimodal blobs
Lakehouse (Delta/Iceberg)	—	Unified BI + ML feature store	Training corpus + RAG + eval + prompt logs
Vector DB (Qdrant/Milvus)	—	Recommendation (rare)	Online RAG (sub-100 ms)
Multimodal (Lance)	—	—	Video/image/audio + embeddings

LLM-era demands (dedup trillion-token, multimodal blobs, embedding versioning, training-data provenance, license/eval tags, prompt+trace logs → Bronze) *thêm tầng* (vector DB, multimodal) và *ép* lakehouse = dataset-of-record cho compliance — chi tiết ở §8 (AI/ML) + §11 (Governance).

_delta_log/



Parquet Files



ACID: Atomicity + Consistency + Isolation + Durability trên S3 · **Concurrency:** optimistic conflict detection · **Metadata layer:** JSON log biến object store → transactional table. **Aha:** reader đọc 003.json trước → danh sách file hợp lệ *atomic* → không bao giờ thấy trạng thái dở-dang (part-003 có, part-004 chưa).

- `df.write.format("delta")` — basic write
- `MERGE INTO ...` = upsert. Workshop 100K ~2×; prod 100M+ → 10–50× vs overwrite
- Schema evolution: `mergeSchema=true`

- Compact → target 128 MB–1 GB
- $10,000 \times 1 \text{ MB} \rightarrow 10 \times 1 \text{ GB}$
- Z-ORDER: sort/co-locate theo cột → min/max stats chặt → skip cả file (3–10× typical; chi tiết §6)

- VACUUM table RETAIN 168 HOURS
- Dài = nhiều cost, audit tốt
- Ngắn = ít cost, mất rollback window
- Regulated 30+ ngày; startup 7 ngày

- Successor cho Z-ORDER: incremental re-cluster
- Không cần rewrite toàn bộ table
- GA Databricks; OSS Delta 3.2+, ổn định trong 4.x

Vấn đề: DELETE 1 row → rewrite cả file (write amplification 1000×).

Cách hoạt động: bitmap đánh dấu rows xóa trong sidecar; reader skip theo bitmap, không rewrite Parquet.

Lợi ích:

- DELETE/UPDATE/MERGE 10–100× nhanh hơn
- GDPR DELETE FROM ... WHERE user_id=X từ giờ → phút
- Compact-on-read; physical removal khi OPTIMIZE

Bật mọi lúc trên table có sẵn: ALTER TABLE ... SET TBLPROPERTIES('delta.enableDeletionVectors'=true)

Use case: downstream consumer cần biết *cái gì thay đổi*, không phải full snapshot.

Cách bật: TBLPROPERTIES
(delta.enableChangeDataFeed = true)

Đọc deltas:

```
spark.read.format("delta").option("readChangeFeed",  
  .option("startingVersion",5).table("silver.users"))
```

Output columns: _change_type (insert / update_pre / update_post / delete) + version + timestamp.

Pattern: Bronze CDF → Silver MERGE → Gold incremental refresh = streaming-like batch.

Kết hợp: Deletion Vectors + CDF + MERGE = canonical CDC sink pattern. Không cần custom apply logic.

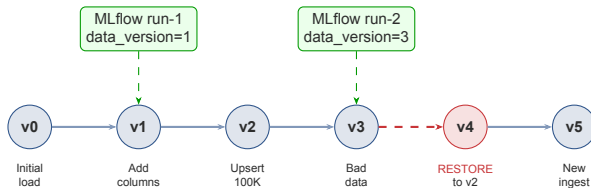
```
# Version-based
df = spark.read.format("delta") \
    .option("versionAsOf", 5).load(path)
# Timestamp-based (point-in-time query)
df = spark.read.format("delta") \
    .option("timestampAsOf", "2025-01-15 00:00:00").load(path)
# Restore (rollback) --- new version, same data as v10
DeltaTable.forPath(spark, path).restoreToVersion(10)
spark.sql("DESCRIBE HISTORY delta.`{}`".format(path)) # audit
```

versionAsOf (số version) · timestampAsOf (point-in-time) · restoreToVersion (rollback) · DESCRIBE HISTORY = audit log built-in. **KHÔNG phải backup** — sống trong VACUUM retention (7 ngày).

- Model reproducibility — pin training set version
- Rollback bad ingestion — instant vs 2+ giờ manual fix
- A/B test datasets — timestamp queries
- Regulatory audit — DESCRIBE HISTORY = compliance log

- Bị giới hạn bởi VACUUM retention (default 7 ngày)
- Schema-incompatible old versions có thể fail
- Khôi phục $\neq$ GDPR DELETE — VACUUM mới xoá vĩnh viễn

Lưu ý: Time travel + schema registry = Data Contracts.



MLflow run_id ↔ Delta table version = reproducible training. Full lineage: data → features → model → deploy.

Feature	Delta Lake	Iceberg	Hudi
ACID Transactions	✓	✓	✓
Time Travel	✓	✓	✓
Deletion Vectors	✓ (2.3+)	✓ (v3)	✓ (MOR)
Hidden Partitioning	×	✓	×
Branching/Tagging	Tag only	✓ + Nessie	×
Multi-engine native	via UniForm	✓ (default)	✓
Row-level Updates	MERGE + DV	MERGE + DV	MOR (fastest)
Ecosystem (origin)	Databricks (2017)	Netflix, Apple (2018)	Uber (2016)

Iceberg metadata: `metadata.json` → `manifest list` → `manifests` → Parquet. Apache XTable convert Delta ↔ Iceberg ↔ Hudi. **MOR** (Merge-On-Read) = ghi delta, merge lúc đọc (ghi nhanh nhất). **UniForm** = 1 bản Parquet, nhiều format metadata. **Hudi 1.x:** secondary index + partial update + non-blocking concurrency.

```
-- Need extra partition column
CREATE TABLE events (
  ts TIMESTAMP,
  ts_day DATE, -- duplicate!
  user_id BIGINT)
PARTITIONED BY (ts_day);
-- User MUST filter ts_day
SELECT * FROM events
WHERE ts_day = '2026-04-01';
-- Forgot ts_day = full scan!
SELECT * FROM events
WHERE ts > '2026-04-01'; -- BAD
```

Vấn đề: ts_day duplicate dữ liệu, user dễ quên filter
→ full scan.

```
-- Partition is a transformation
CREATE TABLE events (
  ts TIMESTAMP,
  user_id BIGINT)
PARTITIONED BY (days(ts));
-- User filters natural column
SELECT * FROM events
WHERE ts > '2026-04-01';
-- Iceberg AUTO-prunes!
```

Transforms: years / months / days /
hours(ts), bucket(N, col), truncate(N,
col).

Partition Evolution: đổi days(ts) →
hours(ts) không cần rewrite data.

Tại sao quan trọng: đa số performance regression trong production lakehouse là vì user *quên partition column*. Hidden partitioning loại bỏ cả class bugs này. **LinkedIn** cite đây là lý do chính migrate Hive → Iceberg.

- Databricks ecosystem heavy
- Spark-first workloads
- Zero friction với Databricks Runtime
- Team quen thuộc Delta API

- Multi-engine: cùng 1 table query từ Spark, Trino, Flink, Snowflake
- Vendor neutrality + partition evolution
- REST Catalog: Polaris (Snowflake OSS), Nessie (git-like branching cho data!)
- GDPR row-level deletes

Adoption 2026: Netflix · Apple · LinkedIn · Adobe at scale. AWS Athena/EMR default. Snowflake native (Polaris).
Iceberg v3 GA H1 2026 (spec 6/2025; Snowflake 7/5, Databricks 28/5 · DBR 18+). Iceberg = de facto open standard.

Engine	Sweet Spot	Scale	Format Support
Spark SQL	ETL, batch ML pipelines	TB–PB	Delta, Iceberg, Hudi (native)
Trino	Federated BI, ad-hoc SQL	GB–PB	Iceberg (native), Delta (connector)
DuckDB	Single-node analytics, dev	MB–100GB	Parquet/Delta/Iceberg via extensions
Photon	Databricks-only fast SQL	TB–PB	Delta, Iceberg
Athena	Serverless ad-hoc on S3	GB–TB	Iceberg native, Delta read-only

- <100 GB, 1 dev: **DuckDB** — zero infra
- ETL Spark-native: **Spark SQL**
- BI multi-source: **Trino**
- AWS-only serverless: **Athena**

Đừng chạy Spark cluster cho 5 GB query — DuckDB nhanh hơn, rẻ gần như 0. Chỉ scale-up khi data > 1 node RAM.

Row-oriented (Avro/JSON)

Row 1	id	name	age	city
Row 2	id	name	age	city
Row 3	id	name	age	city

Columnar (Parquet/ORC)

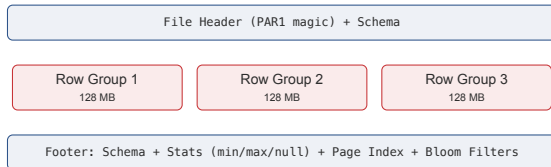
id	name	age	city
v1	v1	v1	v1
v2	v2	v2	v2
v3	v3	v3	v3

Read only age!

SELECT 5/100 columns → đọc ~5% data (Parquet) vs 100% (JSON). Compression: Snappy (default, fastest), ZSTD (3× smaller, hơi chậm), GZIP (chậm nhất).

- Target: 100 MB – 1 GB mỗi partition file
- Partition by **low-cardinality**: date, region
- **Không** partition theo user_id (high-card)
- Over-partition → small files → slow
- Under-partition → large scans → waste IO

- **Partition**: physical directory separation
- **Z-ORDER**: co-locate within files
- Kết hợp: partition by date + Z-ORDER by user_id



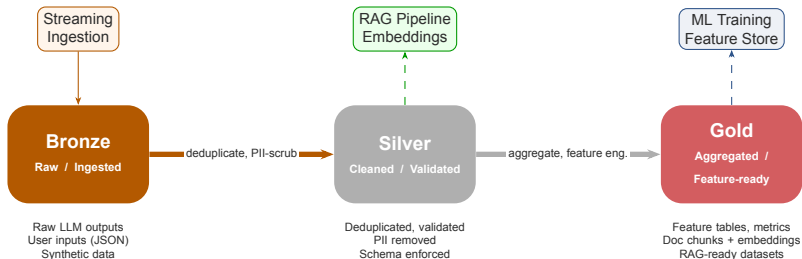
Reader strategy (4 levels of skip): (1) đọc footer trước (~KB) → skip Row Groups bằng min/max stats → (2) đọc Page Index → skip pages → (3) dùng **Bloom Filter** skip values chắc chắn không có → (4) đọc **chỉ columns trong SELECT**.

Compression: Snappy (default, fastest) · ZSTD (3× smaller, hơi chậm) · GZIP (chậm nhất). Quy tắc: Bronze=Snappy (fast write); Gold=ZSTD (slow write OK, fast read).

Practical impact: WHERE user_id=42 trên 1 TB Parquet với bloom filter → đọc ~50 MB. Cùng query CSV/JSON → full 1 TB. **20,000× I/O reduction.**

Change	Safe?	How	Notes
Add column (nullable)	✓	ADD COLUMN; mergeSchema=true	Existing rows = NULL
Add column (with default)	✓	Iceberg v3 native	Delta: backfill MERGE
Rename column	~	Iceberg: RENAME (field ID)	Delta: cần column mapping
Drop column	~	DROP COLUMN	OPTIMIZE physical remove
Type widen (int → bigint)	✓	ALTER COLUMN TYPE	Compatible cast
Type narrow (double → int)	×	— không cho phép	DATA LOSS
Change partition column	~	Iceberg partition evolution	Delta: rewrite
Move column position	✓	ALTER COLUMN ... AFTER	Cosmetic, an toàn

Pattern an toàn: (1) Deploy reader code biết column mới NULL-able → (2) Add column → (3) Backfill → (4) Update writer. Đảo thứ tự = production outage.



Layer	Bronze (raw)	Silver (clean)	Gold (analytics)
Schema	request_id, ts, raw_json	request_id, ts, model, prompt_tokens, completion_tokens, latency_ms, user_id, status	date, model, p50/p95_latency, total_tokens, cost_usd, error_rate
Cardinality	1 row per LLM call	1 row per call (validated)	1 row per (date, model)
Partition	ingest_date	date	date (Z-ORDER model)
Retention	30 ngày	1 năm	5 năm
Consumer	Replay / debug	Feature store, RAG corpus	Dashboards, alerts, FinOps

Quy tắc: Bronze append-only (immutable audit), Silver upsert (MERGE), Gold rebuild-from-Silver (idempotent). Schema rõ ràng mỗi layer = data contract giữa teams. **Tổng quát hoá cho AGENT trajectories:** Bronze = raw steps/tool-call JSON; Silver = parsed (tool, latency, status, cost, trajectory_id); Gold = per-agent success_rate/cost/step_count. Lakehouse = system-of-record; vector store + LangGraph checkpoint = derived memory.

- Raw → processed → model → deploy version chain
- RAG: doc chunks + embeddings lưu Gold
- Embedding gắn doc version = fully traceable
- Pin Delta version trong MLflow run cho reproducibility

- S3 Intelligent-Tiering cho cold Bronze data
- Glacier sau 90 ngày → -60% storage cost
- AWS S3 Tables = managed Iceberg (auto-compaction), v3 từ 11/2025
- Unity/Polaris: fine-grained access + audit

- Inference req/resp → Bronze (raw JSON, 30d)
- Dedup + parse tokens/latency → Silver
- Aggregate (date, model) metrics → Gold

- LLM generate → Bronze (output + prompt)
- Quality filter + MinHash dedup → Silver
- Curated set + license tags → Gold



3 failure modes phổ biến:

- (1) Kafka full → Postgres WAL fills → DB outage. *Fix:* alert on Debezium lag + Kafka retention dài.
- (2) Source schema change breaks pipeline. *Fix:* Schema Registry + mergeSchema=true ở sink.
- (3) Out-of-order events tạo bad updates. *Fix:* MERGE ... WHEN MATCHED AND src.ts > tgt.ts.

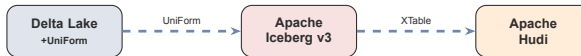
Vietnam context: chuẩn cho fintech VN (MoMo, VNPAY, Cake) — Postgres OLTP → Iceberg/Delta analytics.

End-to-end freshness 1–5 phút, không impact OLTP. (WAL/binlog = Write-Ahead Log: nhật ký thay đổi DB đọc lại được.) **Flink-native streaming** → Apache Paimon (LSM-tree, sub-minute mutable streams, output Iceberg-compatible).

Công ty	Format & Scale	Số liệu công khai
Uber	Hudi · 350 PB	6 T rows/day · 19,500 datasets · freshness 24h → 1h
Netflix	Iceberg + Lance	Atlas query planning 9.6 min → 42 sec · Media Data Lake multimodal
Apple	Iceberg	“Foundation cho lakehouse on all divisions” (MB → PB)
LinkedIn	Iceberg	Migrate Hive → Iceberg vì hidden partitioning + query planning
Shopify	Iceberg + Trino	Open lakehouse, multi-engine BI + ML

- Append-mostly (logs, events) → Delta/Iceberg
- Mutation-heavy (orders, sessions) → Hudi
- Multimodal (video, embeddings) → Lance + Iceberg

MoMo, Zalo, Shopee VN → TB–PB. Pattern chuẩn:
Postgres → Debezium → Kafka → Iceberg/MinIO.
Decree 13 → on-prem MinIO khả thi.



Key events: Databricks acquires Tabular \$1B+ (2024) · Snowflake → Iceberg native + Polaris · Delta 4.0 GA 9/2025 · Iceberg v3 GA H1 2026 (Snowflake 7/5, Databricks 28/5 · DBR 18+); deletion vectors + row lineage + VARIANT + default values + geo. **Vì sao kết thúc:** Delta 4.0 & Iceberg v3 ship CÙNG primitives → chọn theo **catalog + ecosystem**, không phải features.

- AWS Glue — default cho AWS analytics (Athena/EMR/Redshift)
- Unity Catalog — Databricks OSS 2024
- Apache Polaris — top-level Feb 2026
- Project Nessie — git-like branching
- Lakekeeper — Rust, K8s-native

REST Catalog spec = lingua franca 2026. **+Apache Gravitino** (TLP 2025): metadata federation + MCP cho agents.

```
nessie tag create v1-prod
nessie branch create exp-2026
# ...train + evaluate on branch...
nessie merge → main
```

“Model X dùng data nào?” → `nessie tag list`:
trả lời 1 command, thay vì Confluence rotting.

Frontier 2026: DuckLake (v1.0 prod 4/2026) = metadata trong SQL DB thay metadata-files → giải “small-metadata-files”; ecosystem còn mỏng — counter-design, không phải default.

FineWeb (HF): $\sim 15\text{T}$ tokens · per-snapshot MinHash *beat* global dedup · ODC-By (siêu set RedPajama 30T / Dolma 3T)

Lakehouse: Bronze (raw) $\rightarrow$ Silver (dedup) $\rightarrow$ Gold (train-ready); time travel revert dedup sai.

Embedding version = doc_v $\times$ model_v $\rightarrow$ pin via Iceberg/Delta + MLflow run_id. Vector DB = **derived index** (rebuildable), không phải system-of-record (rebuild từ Iceberg authoritative).

- Blob reads tới $\sim 68\times$ nhanh hơn, $\sim 50\%$ nhỏ hơn (Lance v2.2)
- Native HNSW vector index; first-class blobs
- Built-in versioning; Lance-native SQL qua DuckDB (2026)

Netflix Media Data Lake: Parquet $\rightarrow$ Lance cho video frames.

Stack bổ trợ, không thay thế: Iceberg/Delta (tabular) + Lance (multimodal) + Vector DB (online ANN). VN: corpus Wiki-VI/ZNews $\rightarrow$ MinHash dedup trước.

Managed MCP servers expose lakehouse cho agents qua catalog:

- Genie → UC tables (NL query, on-behalf-of-user, UC permissions enforced)
- AI Search → vector index · UC Functions → governed tools
- Snowflake đối xứng: Cortex Analyst MCP → SQL qua semantic view

Agent KHÔNG query raw table — query qua semantic layer (entities, named metrics, joins) → không đoán PK/FK.

AI Gateway = control plane: mỗi tool-call check UC policy *trước*, log *sau* thành UC system tables (= Bronze-tier agent telemetry, nối liền medallion); per-tool enable/disable.

Thực tế text-to-SQL: ReAct ~91% trên familiar DB, *nhưng* Spider 2.0 SOTA chỉ ~45%; rớt 14–40% trên heterogeneous DB. *Mitigation:* semantic layer + query linting + constrained joins.

Lakehouse *passive* (chờ query) → *agentic* (semantic layer = não); cùng 1 ACID table, thêm governance + observability.

Catalog	Origin	Killer feature	Khi nào dùng
AWS Glue	AWS, 2017	Default AWS analytics (Athena/EMR/Redshift)	Đã ở AWS
Unity Catalog	Databricks, OSS 2024	Governance + ML models / feature tables / agent functions	Databricks-native
Apache Polaris	Snowflake, TLP Feb 2026	Vendor-neutral REST	Multi-cloud
Apache Gravitino	Datastrato, TLP 2025	Metadata federation + AI Model Catalog + MCP Server	Heterogeneous + agent
Project Nessie	Dremio, 2020	Git-like branching/tagging	ML versioning
Lakekeeper	OSS 2024 (Rust)	Lightweight, K8s-native	Self-hosted

Standard HTTP API cho Iceberg metadata (createTable, loadTable, commit, listNamespaces).

Lợi: engine (Spark, Trino, Flink, Snowflake, DuckDB) plug-and-play với bất kỳ catalog implement spec. 2026 default.

Catalog-level commit = atomic update nhiều tables.

Use case: Bronze + Silver + Gold update phải all-or-nothing.

Nessie native (git commit); Polaris đang implement.

Where: Bronze ingestion

What: validate raw data

Strength: 50+ expectations

```
expect_column_values_to_be_in_set(
    column="status",
    value_set=["ok",
               "rate_limited", "error"])
```

Where: Silver/Gold transforms

What: structural correctness

Strength: SQL-native

```
# schema.yml
- name: customer_id
  tests:
    - unique
    - not_null
    - relationships:
        to: ref('dim_customers')
```

Where: Prod monitoring

What: continuous, anomaly

Strength: SodaCL DSL

```
checks for gold.metrics:
- row_count >= 1000
- freshness < 1h
- anomaly_score
  < 0.7 for cost_usd
```

Data Contract: schema + constraints + freshness SLA + ownership; run trong CI (pre-merge) và runtime (per-batch). Bề contract → block pipeline.



OpenLineage: OSS standard — Spark/Airflow/dbt/Flink emit lineage events tự động. **Marquez:** reference server (graph DB + UI). Trả lời: “Drop column X → ai bị ảnh hưởng?” · “Gold metric Y sai → truy ngược về Bronze nào?”

Pattern: bật OpenLineage từ ngày 1. Spark: `spark.openlineage.transport.type=http` + Marquez URL.

- **RBAC:** role-based (admin/analyst/eng)
- **ABAC:** attribute-based (region, classification) — Unity/Polaris
- **Row-level security:** dynamic filter
- **Column masking:** hash/redact PII

- Tokenize PII at Bronze landing
- Encrypt at field level (Iceberg v3 native)
- Right-to-forget: DELETE + VACUUM sau 30d grace
- Audit log mọi PII column access

PDPL Law 91/2025/QH15 (hiệu lực 2026-01-01, Decree 356/2025 hướng dẫn) chồng lên **Decree 13/2023** (không thay thế):

- Personal data: basic vs sensitive
- Data residency: sensitive ở VN
- Cross-border: DPIA + CTIA nộp Bộ Công an
- Right-to-forget: 72h SLA

Impact: sensitive data → on-prem MinIO + Iceberg.

- Every read/write → Bronze audit table
- Iceberg: `system.snapshots` table
- Retention 365+ ngày cho regulated

Component (100 TB/tháng)	Snowflake	Iceberg + Trino / S3	Databricks
Storage	\$4,000	\$2,300 (S3)	\$2,300 (S3)
Compute (BI)	\$15,000	\$8,000 (Trino EC2)	\$11,000 (DBU)
Compute (ETL)	\$5,000	\$2,000 (Spark EMR)	\$4,000 (DBU)
Catalog/gov	included	\$500 (Polaris OSS)	included (Unity)
Total	\$24,000	\$12,800 (−47%)	\$17,300 (−28%)

- Per-layer budget: Bronze → Glacier sau 30d
- Auto-OPTIMIZE nightly cron
- Tag clusters: team, purpose, expires_at
- Spot cho ETL, on-demand cho BI
- Top-10 query review hàng tháng

- Cloudera: TPC-DS 24s → 1.8s (13×), storage −36%
- ClickHouse vs Snowflake: ~4× lower TCO
- Netflix Iceberg: planning 9.6min → 42s (14×)
- No OPTIMIZE: 10× *more* expensive than DW

Quy tắc: savings *chỉ* materialize nếu actively optimize. Forget OPTIMIZE → small-file tax giết economics.

#	Anti-pattern	Hậu quả & Fix
1	“Đổ tất cả vào S3” (raw JSON, no schema)	Data swamp → enforce schema từ Bronze; dùng Delta/Iceberg ngay từ đầu
2	Partition theo high-cardinality (vd. user_id)	Triệu partition nhỏ → partition by date/region, Z-ORDER user_id
3	Bỏ qua OPTIMIZE → small-file problem	10K files × 1 MB → query 10× chậm → daily OPTIMIZE cron
4	VACUUM 0 HOURS để “tiết kiệm storage”	Mất time travel + concurrent readers fail → giữ tối thiểu 168h (7 ngày, default)
5	Spark cluster cho query 5 GB	Lãng phí 10× chi phí → DuckDB / Athena cho <100 GB; chỉ scale-up khi > 1 node RAM

80% lakehouse pain trong production = một trong 5 anti-patterns này. Audit checklist trước khi deploy.

- **Small-file problem** — 200 appends → `compact()` + `z_order()` → stats-based pruning: ~1 of ~50 files chứa target user (files-pruned $\geq 10\times$; speedup phụ thuộc RAM, thường $< 3\times$ ở máy nhỏ)
- **Time travel rollback** — inject bad rows → `restoreToVersion(N)` < 30s vs ~2h manual; verify `score < 0` count = 0
- **MERGE upsert** — 100K rows trong < 1s (lightweight); production 100M+ scale → 10–50 $\times$ vs overwrite
- **Audit trail** — `history()` sau RESTORE = 5 versions (gồm cả RESTORE itself)

Lab: VinUni-AI20k/Day18-Track2-Lakehouse-Lab — lightweight (make setup, ~15s) hoặc Spark/Docker (make spark-up).

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 **Lakehouse = ACID + object storage + open formats.** Foundation chung cho 3 era: Traditional, ML, LLM.
- 2 **Format war kết thúc bởi convergence:** Delta 4.0 & Iceberg v3 ship cùng primitives (DV, row lineage, VARIANT) → chọn theo **catalog + ecosystem**, không phải features.
- 3 **Time travel + branching** (Nessie) = “git checkout” cho dataset. **OPTIMIZE + Z-ORDER + Deletion Vectors** bắt buộc cho production.
- 4 **LLM era cần thêm tầng:** Vector DB (RAG), Lance (multimodal), embedding versioning, training data provenance.
- 5 **Production ops trifecta:** Catalog + Data Contracts + Lineage. Bắt từ ngày 1.

Ngày 19: Vector Store & Feature Store

*“SQL trả exact match. AI cần tương tự — semantic search thay đổi mọi thứ. **S3 Vectors** (GA 12/2025): native vector buckets, ~90% rẻ hơn — nhưng vector DB vẫn thắng online sub-100ms.”*

- Hoàn thành Lab 18: 4 notebooks (xem repo URL trên Deliverable)
- Đọc trước: case studies (Netflix, Uber, Apple Iceberg) + Lance multimodal docs
- Cài sẵn Docker + Qdrant image cho Ngày 19 (Vector Store + ANN)

Hỏi & Đáp

Câu hỏi về Lakehouse, Delta/Iceberg, Medallion, Catalog, Data Contracts, hay AI/LLM workloads?



Cảm ơn!

AICB-P2T2 · Ngày 18

Data Lakehouse Architecture

`lms.vinuni.edu.vn` · Lab repo + slides trên LMS